

Counterfactual Data Augmentation With Denoising Diffusion for Graph Anomaly Detection

Chunjing Xiao , Shikang Pang , Xovee Xu , *Graduate Student Member, IEEE*, Xuan Li ,
Goce Trajcevski , *Member, IEEE*, and Fan Zhou , *Member, IEEE*

Abstract—A critical aspect of graph neural networks (GNNs) is to enhance the node representations by aggregating node neighborhood information. However, when detecting anomalies, the representations of abnormal nodes are prone to be averaged by normal neighbors, making the learned anomaly representations less distinguishable. To tackle this issue, we propose an unsupervised counterfactual data augmentation method for graph anomaly detection (CAGAD) that introduces a graph pointer neural network as the heterophilic node detector to identify potential anomalies whose neighborhoods are normal-node-dominant. For each identified potential anomaly, we design a graph-specific diffusion model to translate a part of its neighbors, which are probably normal, into anomalous ones. At last, we involve these translated neighbors in GNN neighborhood aggregation to produce counterfactual representations of anomalies. Through aggregating the translated anomalous neighbors, counterfactual representations become more distinguishable and further advocate detection performance. The experimental results on four datasets demonstrate that CAGAD significantly outperforms strong baselines, with an average improvement of 2.35% on F1, 2.53% on AUC-ROC, and 2.79% on AUC-PR.

Index Terms—Counterfactual data augmentation, graph anomaly detection, graph neural network (GNN), representation learning.

I. INTRODUCTION

ANOMALY detection on a graph aims to detect nodes that present abnormal behaviors and significantly deviate from the majority of nodes [1], [2]. Anomaly detection has numerous high-impact applications in various domains, such as

abnormal user detection [3], [4] and fraud behavior detection [5], [6]. To identify anomalies in graph-structured data, it is of paramount importance to learn expressive and distinguishable node representations by modeling the information from both node attributes and graph topology. Among many approaches that tried different learning methods for anomaly detection, graph neural networks (GNNs) [7], [8] have gained significant attention due to their effectiveness and flexibility [8], [9].

Motivation: One fundamental assumption in GNNs is that similar nodes (w.r.t. node attributes and their labels) have a higher tendency to connect to each other than dissimilar ones. Hence, GNNs typically utilize the message-passing scheme to learn node representations by aggregating node attributes iteratively from their local neighborhoods. However, GNN-based graph anomaly detection models confront a critical over-smoothing issue [10], [11] inherent to GNNs—namely, when aggregating information from the neighborhoods, the representations of anomalous nodes are averaged by normal nodes, as the number of normal nodes is much larger than the number of anomalies. This, in turn, can make the anomaly representations less distinguishable. Hence, the benign neighbors of anomalies might attenuate the suspiciousness of anomalies, resulting in poor detection performance.

Several GNN models are proposed to remedy this issue, mainly falling into two categories: 1) *resampling* methods balance the number of samples by over-sampling the minority class or under-sampling the majority class [12], [13], [14]; and 2) *reweighting* methods assign different weights to different classes or even different samples by cost-sensitive adjustments or metalearning-based methods [15], [16], [17]. Despite considerable performance improvements, they still suffer from certain shortcomings: 1) most of the models rely on massive labeled samples to obtain a good performance. However, labeled anomalies are scarce, and obtaining them is costly and time-consuming, as anomalies are typically rare data instances in most practical applications [1], [18]. 2) These methods mainly manipulate the training data for building a robust classification model. However, for the anomaly detection task, the phenomenon of imbalanced neighbors also exists in practical (testing) data. Ignoring the manipulation of test data might result in significant performance degradation.

Our contributions: To overcome these drawbacks, we propose to manipulate neighbors of anomalous nodes so that the learned anomaly representations are distinguishable from the

Manuscript received 30 April 2023; revised 22 January 2024; accepted 13 May 2024. Date of publication 19 June 2024; date of current version 3 December 2024. This work was supported by the National Natural Science Foundation of China under Grant 62176043 and Grant 62072077. (Corresponding authors: Xovee Xu; Fan Zhou.)

Chunjing Xiao and Shikang Pang are with the School of Computer and Information Engineering and Henan Key Laboratory of Big Data Analysis and Processing, Henan University, Kaifeng 475004, China (e-mail: chunjingxiao@gmail.com; pangsk0604@henu.edu.cn).

Xovee Xu and Fan Zhou are with the University of Electronic Science and Technology of China, Chengdu, Sichuan 610054, China (e-mail: xovee@std.uestc.edu.cn; fan.zhou@uestc.edu.cn).

Xuan Li is with the National Key Laboratory of Fundamental Science on Synthetic Vision, Sichuan University, Chengdu 610065, China (e-mail: lixuanlmw@stu.scu.edu.cn).

Goce Trajcevski is with the Department of Electrical and Computer Engineering, Iowa State University, Ames, IA 50011 USA (e-mail: gocet25@iastate.edu).

Digital Object Identifier 10.1109/TCSS.2024.3403503

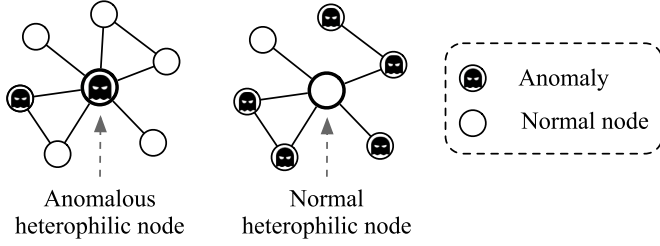


Fig. 1. Nodes with heterophily dominant neighbors.

normal ones. Performing direct manipulation is very difficult since we do not have label information about which nodes are anomalies. Fortunately, the anomaly detection task only has two classes of nodes; thus, it is feasible to identify whether a target node has heterophily dominant neighbors—i.e., most of its neighbors have different class labels from the target node [19], [20]. We refer to these nodes as *heterophilic* nodes, a common occurrence in anomaly detection tasks due to class imbalance. Our intention is to manipulate the neighbors of heterophilic nodes to enhance node representations.

Heterophilic nodes can be either normal or anomalous, as illustrated in Fig. 1. For an anomalous heterophilic node, most of its neighbors are normal nodes, and for a normal heterophilic node, most of its neighbors are anomalies [19]. Since the labels of heterophilic nodes are unknown, we treat anomalous and normal heterophilic nodes equally, i.e., for each heterophilic node, we translate a part of its neighbors into anomalous ones and involve these translated nodes for neighborhood aggregation. Specifically, for anomalous heterophilic nodes, translating a part of their neighbors (which are probably normal) into anomalies would enhance the neighborhood aggregation for learning distinguishable node representations. Likewise, for normal heterophilic nodes, translating their neighborhoods (which are probably anomalous) into anomalies would have only a small impact on the neighborhood aggregation since we do not change the labels of these neighborhoods. The selection of translated nodes can be guided by a ranking strategy according to their similarities to the heterophilic node.

Based on this idea, we present a counterfactual data augmentation for graph anomaly detection (CAGAD) that follows a *detection* $\rightarrow$ *translation* $\rightarrow$ *representation* scheme: 1) we first introduce a graph pointer neural network (GPNN) as the heterophilic node detector to identify nodes with heterophily dominant neighbors. 2) Then, for each heterophilic node, we translate a part of its neighbors into anomalous ones using a probabilistic anomaly generator, which is powered by a denoising diffusion probabilistic model (DDPM) [21]. The generator can extract distinct features of anomalies, and these features are regarded as the conditions to be imposed on the generative process to create new anomalous neighbors for the heterophilic node. 3) Finally, the generated anomalous neighbors are involved in a counterfactual GNN that is built by incorporating the anomaly generator into a graph attention network (GAT). Counterfactual GNN aggregates the information of new unseen neighbors and produces counterfactual node representations.

The detection, translation, and aggregation processes in CAGAD are performed in an unsupervised way without labeled data. Unlike many of the traditional anomaly detection methods, our proposed CAGAD can be applied to test data for enhancing the node representations and will not impact the nodes' identification information (i.e., the generated counterfactual representations still have the same labels as the original ones, as changing a few neighbors or edges will not impact the node's identification (label) information [22], [23]). Through this scheme, our model learns more distinguishable representations for anomalous nodes and alleviates the over-smoothing issue of GNNs in graph anomaly detection. The main contributions of our work are as follows.

- 1) We propose a novel CAGAD, which can produce counterfactual representations by aggregating unseen neighbors to enhance the representations of anomalies in an unsupervised learning way.
- 2) We design a probabilistic anomaly generator powered by denoising diffusion models. It iteratively extracts distinct features of anomalies, which are involved in the generative process to transfer neighbors into anomalous ones.
- 3) We propose a counterfactual GNN to generate counterfactual node representations by aggregating the translated neighbors rather than the original ones, which can alleviate the smooth aggregation problem of GNNs.
- 4) Extensive experiments on four public anomaly detection datasets showed that CAGAD significantly improves the detection performance compared to state-of-the-art baselines.¹

The rest of this article is organized as follows. Section II describes the problem formulation and necessary backgrounds. Section III presents the details of our CAGAD model. Section IV compares our model with baseline approaches on four datasets. Section V reviews the literature related to anomaly detection, graph data augmentation, and diffusion models. Finally, we conclude this work with future directions in Section VI.

II. PRELIMINARIES

We now formulate the graph anomaly detection problem and give the necessary background information on the DDPMs, which are used to generate new anomalous neighbors.

A. Problem Formulation

Following the commonly used notations, we use calligraphic fonts, bold lowercase letters, and bold uppercase letters to denote sets (e.g., $\mathcal{V}$), vectors (e.g., $\mathbf{x}$), and matrices (e.g., $\mathbf{X}$), respectively. In general, an attributed network can be represented as $G = (\mathcal{V}, \mathcal{E}, \mathbf{X})$, where $\mathcal{V} = \{v_1, v_2, \dots, v_n\}$ denotes the set of nodes, $\mathcal{E} = \{e_1, e_2, \dots, e_m\}$ denotes the set of edges, and $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\} \in \mathbb{R}^{n \times h}$ denotes the h -dimensional attributes of n nodes. A binary adjacency matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ is the structural information of the attributed network, where $\mathbf{A}_{i,j} = 1$ if there is a link between nodes v_i and v_j ; $\mathbf{A}_{i,j} = 0$ otherwise.

¹Source code is released at <https://github.com/ChunjingXiao/CAGAD>

TABLE I
LIST OF NOTATIONS

Notation	Description
v	The node vector of a graph
u	The attention score of a node
l	A hidden layer in the GATs
η/α	A given threshold
$\mathbf{h}/\mathbf{z}/\mathbf{d}/\mathbf{e}$	The feature vector of a node
$\mathcal{V}$	The set of nodes of a graph
$\mathcal{E}$	The set of edges of a graph
G	A graph $G = (\mathcal{V}, \mathcal{E}, \mathbf{X})$
$\mathbf{X}$	The node feature matrix that G may have
$\mathbf{A}$	The adjacent matrix of a graph
$\mathbb{R}$	The set of real numbers
$\mathbf{H}/\mathbf{Z}$	The feature matrix of a graph

Accordingly, the problem of anomaly detection on an attributed graph is defined as follows.

Problem 1 (Anomaly Detection): Let $\mathcal{V}_a$ and $\mathcal{V}_n$ be two disjoint subsets from $\mathcal{V}$, where $\mathcal{V}_a$ refers to all the anomalous nodes and $\mathcal{V}_n$ denotes all the normal nodes. Graph-based anomaly detection is to classify unlabeled nodes in G into the normal or anomalous categories given the information of the graph structure $\mathbf{A}$, node features $\mathbf{X}$, and partial node labels from $\mathcal{V}_a$ and $\mathcal{V}_n$.

Usually, there are far more normal nodes than anomalous nodes, $|\mathcal{V}_a| \ll |\mathcal{V}_n|$, thus graph-based anomaly detection can be regarded as an extremely imbalanced binary node classification problem. The main difference is that anomaly detection focuses more on the unusual and deviated patterns in the dataset. A summary of the symbols used in this article is presented in Table I.

B. DDPMs

DDPMs [21], [24] are a class of generative models that show superior performance in unconditional image generation compared to GANs [25], [26]. It learns a Markov Chain that gradually converts a simple distribution (e.g., isotropic Gaussian) into a data distribution. The generative process learns the reverse of the DDPM's forward (diffusion) process: a fixed Markov Chain that gradually adds noise to data. Here, each step in the forward process is a Gaussian translation

$$q(\mathbf{z}^t | \mathbf{z}^{t-1}) := N(\mathbf{z}^t; \sqrt{1 - \beta_t} \mathbf{z}^{t-1}, \beta_t \mathbf{I}) \quad (1)$$

where $\beta_1, \dots, \beta_T$ is a fixed variance schedule rather than learned parameters [21]. Equation (1) is a process finding $\mathbf{z}^t$ by adding a small Gaussian noise to the latent variable $\mathbf{z}^{t-1}$. Given clean data $\mathbf{z}^0$, sampling of $\mathbf{z}^t$ can be expressed in a closed form

$$q(\mathbf{z}^t | \mathbf{z}^0) := N(\mathbf{z}^t; \sqrt{\bar{\alpha}_t} \mathbf{z}^0, (1 - \bar{\alpha}_t) \mathbf{I}) \quad (2)$$

where $\alpha_t := 1 - \beta_t$ and $\bar{\alpha}_t := \prod_{s=1}^t \alpha_s$. Therefore, $\mathbf{z}^t$ is expressed as a linear combination of $\mathbf{z}^0$ and ϵ

$$\mathbf{z}^t = \sqrt{\bar{\alpha}_t} \mathbf{z}^0 + \sqrt{1 - \bar{\alpha}_t} \epsilon \quad (3)$$

where $\epsilon \sim N(0, \mathbf{I})$ has the same dimensionality as data $\mathbf{z}^0$ and latent variables $\mathbf{z}^1, \dots, \mathbf{z}^T$.

Since the reverse of the forward process $q(\mathbf{z}^{t-1} | \mathbf{z}^t)$ is intractable, DDPM learns parameterized Gaussian transitions $p_\theta(\mathbf{z}^{t-1} | \mathbf{z}^t)$. The generative (or reverse) process has the same functional form [24] as the forward process, and it is expressed as a Gaussian transition with learned mean and fixed variance [21]

$$p_\theta(\mathbf{z}^{t-1} | \mathbf{z}^t) = N(\mathbf{z}^{t-1}; \mu_\theta(\mathbf{z}^t, t), \sigma_t^2 \mathbf{I}). \quad (4)$$

Further, by decomposing μ_θ into a linear combination of $\mathbf{z}^t$ and the noise approximator ϵ_θ , the generative process is expressed as

$$\mathbf{z}^{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{z}^t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{z}^t, t) \right) + \sigma_t \epsilon \quad (5)$$

which suggests that each generation step is stochastic. Here, ϵ_θ represents a neural network with the same input and output dimensions, and the noise predicted by the neural network ϵ_θ in each step is used for the denoising process in (5).

C. GPNN

The GPNN [20] aims to select the most relevant and valuable neighboring nodes, which constructs a new sequence ranked by relation to the central node. GPNN takes a node sequence consisting of the neighbors of a given central node as input and outputs an ordered sequence according to the relationship with the central node. This model utilizes a sequence-to-sequence architecture based on LSTMs for the pointer network to order the sequence.

Specifically, GPNN is composed of an *encoder* and a *decoder*. The encoder generates hidden states for the input sequence. At each time step i , the feature vector of node $\hat{\mathbf{x}}_i$ is fed into the encoder, the hidden state is denoted as

$$\mathbf{e}_i = \tanh(\mathbf{W}[\mathbf{e}_{i-1}, \hat{\mathbf{x}}_i]) \quad (6)$$

where $\mathbf{e}_0$ is initialed to zero vector. After L time steps, GPNN obtains L hidden states of input sequence and combines them into a content vector $\mathbf{E} = \{\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_L\}$ that records the information of entire sequence of L nodes.

The decoder then selects nodes with attention scores among the L nodes. At each output time i , the hidden state of decoder $\mathbf{d}_i$ is obtained by

$$\mathbf{d}_i = \tanh(\mathbf{W}[\mathbf{d}_{i-1}, \hat{\mathbf{x}}_{c_{i-1}}]) \quad (7)$$

where $\mathbf{d}_0 = \mathbf{e}_L$ is the output hidden state from the encoder and c_{i-1} is the index of the selected node at the time step $i - 1$. GPNN computes the attention vectors over the input sequence as following:

$$u_j^i = \beta^T \tanh(\mathbf{W}_1 \mathbf{e}_j + \mathbf{W}_2 \mathbf{d}_i), \quad j \in (1, 2, \dots, L) \quad (8)$$

$$p(c_i | c_1, c_2, \dots, c_{i-1}, s) = \text{softmax}(\mathbf{u}_i) \quad (9)$$

where β^T , $\mathbf{W}_1$, and $\mathbf{W}_2$ are learnable parameters of the decoder model, and softmax normalizes the vector $\mathbf{u}_i$ (of length L) to be an output distribution over the L nodes of the input sequence. With the probability distribution $\mathbf{u}_i$, GPNN uses u_j^i as pointers to select the i th node of output sequence, until all top- k nodes

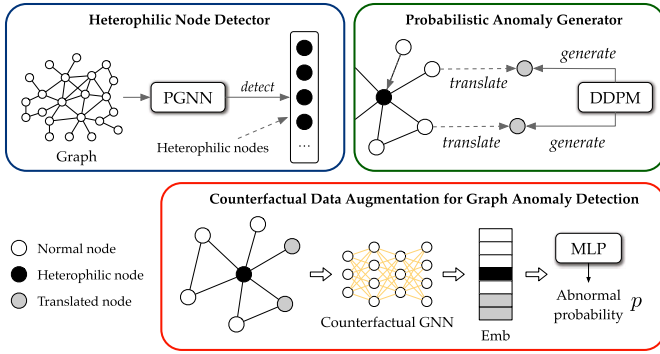


Fig. 2. Sketch of CAGAD. Heterophilic nodes are defined as most of their neighbors having different properties or labels from themselves.

are selected step by step. After k output time steps, GPNN obtains the sequence of top- k relevant nodes as: $\{\hat{\mathbf{x}}_{c1}, \hat{\mathbf{x}}_{c2}, \dots, \hat{\mathbf{x}}_{ck}\} (\in \mathbb{R}^{k \times d})$, which is ranked with the output order.

III. CAGAD: DETAILED METHODOLOGY

We now discuss the CAGAD framework in detail. We first present the heterophilic node detector that identifies nodes with heterophily dominant neighbors in an unsupervised manner. Then, we introduce the DDPM-based anomaly generator, which aims to translate the source embedding into the ones with anomalous labels. Last, we incorporate the generator into GNNs to manufacture counterfactual augmented representations by aggregating generated unseen neighbor embeddings. These augmented representations will be used for enhancing anomaly detection. A sketch of CAGAD is shown in Fig. 2.

A. Heterophilic Node Detector

We first present the heterophilic node detector to identify the nodes whose neighbors are heterophily dominant without requiring labeled data. The GPNN [20] is used to build the detector, which calculates the heterophily degree for heterophilic node detection via attention scores. Here, we aim to distinguish crucial information from distant nodes while filtering out irrelevant or noisy ones in the nearest neighborhood. To this end, we leverage a pointer network to compute attention vectors (scores) and then select the most relevant nodes from multihop neighborhoods according to these scores. Since the attention scores can denote the relevant relationship with the target node [20], we use them to identify heterophilic nodes: a node is regarded as the heterophilic one if most of its neighbors have lower attention scores than a given threshold, say η . The optimal threshold η can be determined by the existing training data.

Specifically, we first construct a node sequence for each target node that contains neighbors within a given hop, such as one hop. As the number of neighbors varies from node to node, we set a fixed maximum length L of the sequence and stop sampling when we meet this limitation. Note that truncating neighbors will not impact detection results since the heterophilic nodes are determined by the ratios instead of absolute node number. When some nodes have a small number of

neighbors, we duplicate one neighbor repeatedly until reaching L and, after obtaining the attention scores, the duplicated ones will be deleted. Then, we apply a GCN layer [27] to compute local embeddings, which captures the local information of each node. With the input nodes feature $\mathbf{X}$, the output embedding is denoted as

$$\hat{\mathbf{X}} = \text{GCN}(\mathbf{X}) \quad (10)$$

where we embed the feature vectors into hidden representations.

Next, we use a pointer network to calculate the attention scores for the node sequence. A sequence-to-sequence architecture based on LSTMs is adopted for the pointer network, composing an encoder and a decoder. For the target node v_i and one of its neighbors v_j , the attention score u_j^i is computed as follows:

$$u_j^i = \beta^T \tanh(\mathbf{W}_1 \mathbf{e}_j + \mathbf{W}_2 \mathbf{d}_i) \quad (11)$$

where β^T , $\mathbf{W}_1$, and $\mathbf{W}_2$ are learnable parameters of the model and $\mathbf{e}_j$ and $\mathbf{d}_i$ are the hidden states of the encoder and decoder, respectively (see Section II-C for details).

Then, we are able to compute the desired heterophily degree h_d to determine whether the target node is a heterophilic one via

$$h_d = |\mathcal{V}_i^\eta| / |\mathcal{V}_i^S| \quad (12)$$

where $\mathcal{V}_i^\eta = \{v : u_j^i < \eta\}$ is the set of the neighbors whose attention scores are less than the threshold η , and $\mathcal{V}_i^S$ is the set of v_i 's neighbors that are included in the sequence. Obviously, h_d represents the percentage of heterophilic neighbors among all neighbors of the target node. For example, $h_d = 60\%$ means that 60% of its neighbors have different class labels from it. If heterophily degree h_d of a node is greater than a given value α , that node is considered heterophilic.

Based on the above, we can extract all the heterophilic nodes of a graph. For each of them, we select a given fraction of neighbors with lower attention scores and translate them into anomalous nodes. These generated anomalous neighbors are used to replace the original ones for neighborhood aggregation. If the heterophilic target node is anomalous, most of its neighbors should be normal. Some normal neighbors will be replaced with generated anomalous ones. This replacement can make the representations more distinguishable. If the heterophilic target node is normal, most of its neighbors should be anomalous. Again, some anomalous neighbors will be replaced with generated anomalous ones. This replacement will hardly influence the target node, since the number of anomalous neighbors remains unchanged. Thus, manipulating heterophilic node neighbors will not impact normal node representations but can benefit anomaly representations to boost performance.

B. DDPM-Based Anomaly Generator

Having identified nodes with heterophily dominant neighbors, we now introduce how the node embeddings are translated into anomalous ones. Inspired by the superiority of diffusion models in image generation and translation [28], [29], [30], we design a graph-specific probabilistic diffusion model as

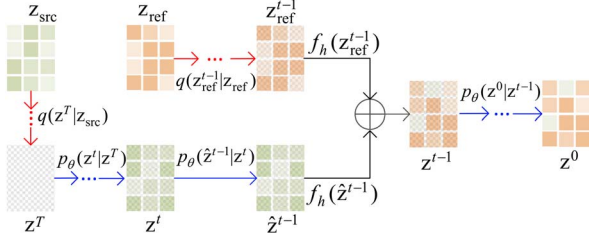


Fig. 3. Anomalous generator G_{ano} : red arrows $\rightarrow$ indicate forward diffusion, and blue ones $\rightarrow$ refer to the reverse diffusion; $\oplus$ is the concatenation operation.

the anomaly embedding generator. This model can iteratively extract distinct features from anomalies and then exert them on the generative process to manufacture anomalous nodes. This generator takes a source embedding $\mathbf{z}_{\text{src}}$ without label information and a reference embedding $\mathbf{z}_{\text{ref}}$ with the anomalous label as inputs and translates the source embedding into an anomalous one. The source embedding can be either normal or anomalous one. Here, the reference embedding can be obtained by using the aforementioned GCN layer [27] or a trained GNN-based classifier.

Fig. 3 presents the framework of the anomaly embedding generator G_{ano} . The generator first adopts the forward process to add noise into $\mathbf{z}_{\text{src}}$ to form a prior $\mathbf{z}^T$. Then, the prior $\mathbf{z}^T$ is fed into the reverse diffusion process to generate a clean embedding through gradual denoising, i.e., $\mathbf{z}^T \rightarrow \mathbf{z}^t \rightarrow \hat{\mathbf{z}}^{t-1} \rightarrow \mathbf{z}^{t-1} \rightarrow \mathbf{z}^0$. During this denoising process, the distinct features of anomalous nodes are extracted from $\mathbf{z}_{\text{ref}}$ and iteratively injected into the latent variable $\hat{\mathbf{z}}^{t-1}$. In this way, the generated embedding $\mathbf{z}^0$ not only has the anomalous label of the reference embedding $\mathbf{z}_{\text{ref}}$ but also keeps the characteristics of the source embedding $\mathbf{z}_{\text{src}}$.

Concretely, we first utilize the forward process $q(\mathbf{z}^t|\mathbf{z}^0)$ to generate a prior $\mathbf{z}^T$ by adding noise

$$\mathbf{z}^T = \sqrt{\bar{\alpha}_T} \mathbf{z}_{\text{src}} + \sqrt{1 - \bar{\alpha}_T} \epsilon \quad (13)$$

where $\bar{\alpha}_T = \prod_{t=1}^T (1 - \beta_t)$ and $\beta_1, \dots, \beta_T$ is a fixed variance schedule [21]. Then, we feed the prior $\mathbf{z}^T$ into the reverse diffusion process and exert the condition c_{ano} on this reverse process to generate an embedding with the anomalous label. We approximate the Markov transition under the condition c_{ano} as follows:

$$p_\theta(\mathbf{z}^{t-1}|\mathbf{z}^t, c_{\text{ano}}) \approx p_\theta(\mathbf{z}^{t-1}|\mathbf{z}^t, f_h(\mathbf{z}^{t-1}) = f_h(\mathbf{z}_{\text{ref}}^{t-1})) \quad (14)$$

where $\mathbf{z}_{\text{ref}}^{t-1}$ is sampled by (3) and $f_h(\cdot)$ is a high-pass filter.

In each Markov transition, (14) tries to incorporate the high-frequency information extracted from $\mathbf{z}_{\text{ref}}$ by $f_h(\cdot)$ into the generated embedding. Since anomalies tend to have different features from their neighbors (high-frequency information), and normal nodes tend to share common features with their normal neighbors (low-frequency information) [10], [11] high-frequency information can be regarded as the distinct features of the anomalies. Hence, we utilize the high-pass filter $f_h(\cdot)$ to extract it from $\mathbf{z}_{\text{ref}}^{t-1}$ (corrupted $\mathbf{z}_{\text{ref}}$), and inject this information into the generated embedding. As a result, the embedding

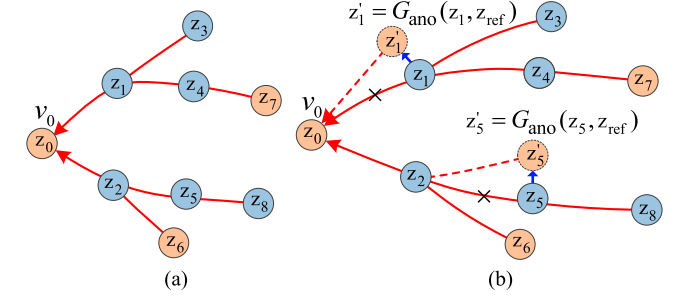


Fig. 4. Original GNN versus counterfactual GNN. (a) Original aggregation. (b) Aggregation via counterfactuals.

generated by (14) will be inclined to possess the anomalous information (label). Since the transition of (14) starts with the prior $\mathbf{z}^T$ (derived from $\mathbf{z}_{\text{src}}$), the generated embedding also has the characteristics of source embedding $\mathbf{z}_{\text{src}}$.

According to (14), in each transition from $\mathbf{z}_t$ to $\mathbf{z}_{t-1}$, the distinct features of anomalies $f_h(\mathbf{z}_{\text{ref}}^{t-1})$ are extracted from the corrupted reference embedding $\mathbf{z}_{\text{ref}}^{t-1}$ and then injected into the latent variable. To this end, we first adopt the forward process (3) to compute $\mathbf{z}_{\text{ref}}^{t-1}$ from $\mathbf{z}_{\text{ref}}$, and then adopt the reverse process (4) to calculate the latent variable $\hat{\mathbf{z}}^{t-1}$ from $\mathbf{z}^t$. Since high-filter operation $f_h(\cdot)$ maintains the dimensionality of the input, we refine the latent variables by matching $f_h(\hat{\mathbf{z}}^{t-1})$ of $\hat{\mathbf{z}}^{t-1}$ with that of $\mathbf{z}_{\text{ref}}^{t-1}$ as follows:

$$\begin{aligned} \mathbf{z}_{\text{ref}}^{t-1} &\sim q(\mathbf{z}_{\text{ref}}^{t-1}|\mathbf{z}_{\text{ref}}) \\ \hat{\mathbf{z}}^{t-1} &\sim p_\theta(\hat{\mathbf{z}}^{t-1}|\mathbf{z}^t) \\ \mathbf{z}^{t-1} &= \hat{\mathbf{z}}^{t-1} + \gamma (f_h(\mathbf{z}_{\text{ref}}^{t-1}) - f_h(\hat{\mathbf{z}}^{t-1})) \end{aligned} \quad (15)$$

where γ is a weight parameter to adjust the importance of the high-frequency information. The matching operation by (15) ensures the condition c_{ano} in (14), which further enables the conditional generation based on DDPM. In this way, by injecting distinct features of anomalies into the latent variable in the generative process, the generated embedding can possess the anomalous label of $\mathbf{z}_{\text{ref}}$. At the same time, the input of the generative process is the prior derived from $\mathbf{z}_{\text{src}}$; thus, the generated embedding still has characteristics of $\mathbf{z}_{\text{src}}$.

C. Counterfactual GNN

We here incorporate the anomaly embedding generator G_{ano} into a GAT to build the counterfactual GNN, which can generate effective augmented data by involving generated anomaly embeddings into the neighborhood aggregation process. Fig. 4 presents the neighborhood aggregation process. For the original GNN in Fig. 4(a), the representation of the target node v_0 is obtained by aggregating the embeddings of its neighbors directly. For the counterfactual GNN in Fig. 4(b), before neighborhood aggregation, we first select the node with heterophily dominant neighbors (such as v_0) and translate a part of its neighbors (e.g., v_1 and v_5) into anomalous ones (e.g., v'_1 and v'_5) by using G_{ano} . Then, the generated nodes are used to replace the original neighbors for GNN neighborhood aggregation to compute the representation $\mathbf{z}_0$ of the target node v_0 .

A representation computed by the counterfactual GNN is regarded as counterfactual data because its generation process involves unseen nodes. Moreover, the counterfactual representation will have the same label as the one calculated by the original GNN, as changing a few edges/neighbors will not greatly impact the node identification (label) information [22], [23]. Hence, this counterfactual GNN can be applied to test data to obtain better node representations.

We use the GATs [9] as the backbone of the counterfactual GNN, which introduces the masked attention mechanism to represent the importance of different adjacent nodes. Formally, in each layer $l - 1$, the node v_i integrates the features of neighboring nodes to obtain representations of layer l via

$$\mathbf{h}_i^{(l)} = \sigma \left(\sum_{j \in \mathcal{V}_i \cup \mathcal{V}'_i \cup \{v_i\}} a_{ij} \mathbf{W} \cdot \mathbf{h}_j^{(l-1)} \right) \quad (16)$$

where σ refers to a nonlinear activation function (e.g., ReLU), $\mathcal{V}_i$ is the set of neighbors for v_i (excluding the neighbors selected as the source ones), $\mathcal{V}'_i$ is the set of neighbors corresponding to the embeddings generated by G_{ano} , and a_{ij} is the attention coefficient between node v_i and node v_j , which can be computed as

$$a_{ij} = \frac{\exp(\sigma(a^T[\mathbf{W}\mathbf{h}_i^{(l)} \oplus \mathbf{W}\mathbf{h}_j^{(l)}]))}{\sum_{k \in \mathcal{V}_i \cup \mathcal{V}'_i \cup \{v_i\}} \exp(\sigma(a^T[\mathbf{W}\mathbf{h}_i^{(l)} \oplus \mathbf{W}\mathbf{h}_k^{(l)}]))} \quad (17)$$

where $\oplus$ is the concatenation operation and attention vector a is a trainable weight vector that assigns importance to different neighbors of node v_i , allowing the model to highlight the features of the important neighboring node that is more task-relevant.

To incorporate high-order neighborhood, multiple layers are adopted to build the graph-attentive encoder

$$\begin{aligned} \mathbf{h}_i^{(1)} &= \sigma \left(\sum_{j \in \mathcal{V}_i \cup \mathcal{V}'_i \cup \{v_i\}} a_{ij}^{(1)} \mathbf{W}^{(1)} \cdot \mathbf{x}_j \right) \\ &\dots\dots \\ \mathbf{z}_i &= \sigma \left(\sum_{j \in \mathcal{V}_i \cup \mathcal{V}'_i \cup \{v_i\}} a_{ij}^{(L)} \mathbf{W}^{(L)} \cdot \mathbf{h}_j^{(l-1)} \right) \end{aligned} \quad (18)$$

where $\mathbf{z}_i$ is the latent representation of node v_i . In this way, the graph-attentive encoder is able to map the learned node representations by capturing the nonlinearity of topological structure and node attributes.

The aggregated representations $\mathbf{z}_i$ are then fed into another MLP with a Sigmoid function to compute the abnormal probability p_i . The weighted cross-entropy loss is then used for the model training

$$\mathcal{L} = \sum_i (\varphi y_i \log(p_i) + (1 - y_i) \log(1 - p_i)) \quad (19)$$

where φ is the proportion of anomaly labels ($y_i = 1$) to normal labels ($y_i = 0$).

This model can improve the detection performance by considering augmented counterfactual representations because the

counterfactual representations of anomalies are enhanced by involving information of generated anomalous neighbors. The proposed data augmentation method scarcely requires labeled data since its main components—the heterophilic node detector and DDPM-based generator—are unsupervised. At the same time, this method can also be used to enhance node representations of test data because it requires no labels for the test data, and the computed counterfactual representations have the same labels as the source ones.

D. Discussions

In this section, we show the connection between our model and GNNs as well as causal representation learning on graphs.

1) *Relation to GNNs*: GNN has been a mainstream technique for graph anomaly detection due to its ability to learn expressive node representations by aggregating node's local neighborhoods [8], [31]. One of the key components in CAGAD is the counterfactual GNN which manipulates the neighborhood aggregation to produce counterfactual node representations. In the following, we present a detailed comparison between the vanilla GNN and the proposed counterfactual GNN.

a) *The original GNN*: Given the adjacency matrix $\mathbf{A}$ indicating connections between nodes in $\mathcal{V}$ and the node attributes $\mathbf{X}$, the original GNN computes the node representations by aggregating features from the local neighborhoods iteratively

$$\begin{aligned} \mathbf{H}^1 &= \text{GNN}^1(\mathbf{A}, \mathbf{X}) \\ \mathbf{H}^2 &= \text{GNN}^2(\mathbf{A}, \mathbf{H}^1) \\ &\dots \\ \mathbf{Z} &= \text{GNN}^L(\mathbf{A}, \mathbf{H}^{L-1}) \end{aligned} \quad (20)$$

where GNN^L is the last aggregation layer and $\mathbf{Z} = [\mathbf{z}_1, \dots, \mathbf{z}_n]$ is the final learned node representations from the GNN. The learned representations $\mathbf{Z}$ can be adopted to identify anomalies from normal nodes by classification-based models or reconstruction-error-based methods.

b) *The counterfactual GNN*: In the proposed model, according to the detection results obtained by the heterophilic node detector (whether nodes are the heterophilic ones), node representations $\mathbf{Z} = [\mathbf{z}_1^o, \dots, \mathbf{z}_m^o, \mathbf{z}_{m+1}^h, \dots, \mathbf{z}_n^h]$ are split into two groups: one for nonheterophilic node representations $\mathbf{Z}^o = [\mathbf{z}_1^o, \dots, \mathbf{z}_m^o]$ and another for heterophilic node representations $\mathbf{Z}^h = [\mathbf{z}_{m+1}^h, \dots, \mathbf{z}_n^h]$. For the nonheterophilic nodes, their representations $\mathbf{Z}^o$ are still computed by using the original GNN (20).

On the other hand, for the heterophilic nodes, their representations $\mathbf{Z}^h$ are learned by the counterfactual GNN, where a part of neighbors will be translated into the new ones with anomalous labels. The aggregating process is expressed as

$$\begin{aligned} \mathbf{H}^1 &= \text{GNN}^1(\mathbf{A}, \mathbf{X}) \\ \hat{\mathbf{H}}_s^1 &= G_{\text{ano}}(\mathbf{H}_s^1, \mathbf{Z}_{\text{ref}}) \\ \mathbf{H}^2 &= \text{GNN}^2(\mathbf{A}, (\mathbf{H}^1 - \mathbf{H}_s^1) \cup \hat{\mathbf{H}}_s^1) \\ &\dots \\ \mathbf{Z}^h &= \text{GNN}^L(\mathbf{A}, \mathbf{H}^{L-1}) \end{aligned} \quad (21)$$

where $\mathbf{H}_s^1$ is the embedding set of selected neighbors to be translated into anomalous ones. The counterfactual GNN still takes the adjacency matrix $\mathbf{A}$ and the node attributes $\mathbf{X}$ as inputs. But during iterative aggregation, a part of the neighbor embeddings of the heterophilic nodes are translated into the ones with anomalous labels by our designed generator G_{ano} , and these generated embeddings are involved for learning node representations. By manipulating the neighborhood aggregation, we can manufacture counterfactual representations that are more distinguishable from normal node representations and subsequently improve the detection performance.

2) *Relation to Causal Representation Learning on Graph*: Causal representation learning aims to learn a representation exposing the causal relations that are invariant under different interventions. Generating counterfactual data are an effective way to remove spurious correlations, help learn better representations, and further improve the classification performance. Existing data augmentation methods for graph data mainly focus on manipulating edges or node attributes, and then they utilize GNNs to generate counterfactual augmented data. Assuming that $\hat{\mathbf{A}}$ and $\hat{\mathbf{X}}$ denote the manipulated adjacency matrix and attribute matrix individually, general augmentation methods adopt the plain GNN to produce augmented data by considering the manipulated adjacency matrix: $\mathbf{Z} = \text{GNN}(\hat{\mathbf{A}}, \mathbf{X})$, or the manipulated attribute matrix: $\mathbf{Z} = \text{GNN}(\mathbf{A}, \hat{\mathbf{X}})$, or both: $\mathbf{Z} = \text{GNN}(\hat{\mathbf{A}}, \hat{\mathbf{X}})$.

By contrast, our model CAGAD differentiates them in two aspects: 1) instead of simply changing edges and attributes, CAGAD manipulates the neighborhood aggregation process to produce counterfactual augmented data, which can generate more flexible augmented data; and 2) unlike these methods, which principally focus on enhancing training data, our model can be applied to test data in boosting representation of anomalies.

IV. EXPERIMENTAL EVALUATION

In this section, we conduct empirical evaluations to demonstrate the efficacy of our approach concerning anomaly detection performance, and the impact of the translated neighbor number and the counterfactual augmentation technique.

A. Experimental Settings

1) *Datasets*: We perform experiments on four datasets introduced in Table II. *PubMed* [32] is a citation network of biomedical science, where the nodes and edges denote the scientific publication and citation connections between publications. *T-Finance* [10] is a transaction network. The nodes are unique anonymized accounts with 10-D features related to registration days, logging activities, and interaction frequency. The edges in the graph are transaction records of accounts. *Amazon* [33] is a review network, where nodes represent users, and the edges indicate that two users have reviewed or rated the same product. *YelpChi* [34] is a review network collected from yelp.com, where nodes represent reviewers and links suggest two reviewers have commented on the same product.

2) *Baselines*: We compare the proposed framework with three types of baselines with different techniques, including

TABLE II
STATISTICS OF DATASETS

Datasets	# Node	# Edge	# Feature	Anomaly (%)
PubMed	19 717	44 338	500	20.81%
T-finance	39 357	21 222 543	10	4.58%
Amazon	11 944	4 398 392	25	6.87%
YelpChi	45 954	3 846 979	32	14.53%

general GNN models [8], [9], [35], [36], graph data augmentation approaches [22], [37], and state-of-the-art methods for graph-based anomaly detection [10], [12], [13], [14]. *GAT* [9] is a graph attention network that employs the attention mechanism for neighbor aggregation. *GIN* [8] is a GNN model connecting to the Weisfeiler–Lehman graph isomorphism test. *GraphSAGE* [35] is a GNN model based on a fixed sample number of the neighboring nodes. *GWNN* [36] is a graph wavelet neural network using heat kernels to generate wavelet transforms. *LA-GNN* [37] is an efficient data augmentation strategy to generate more features in the local neighborhood to enhance the expressive power of GNNs. *GCA* [22] is a contrastive framework that performs data augmentation on both topology and attribute levels to enhance node representations. *GraphConsis* [14] is a heterogeneous GNN that tackles context, feature, and relation inconsistency problems in graph anomaly detection. *CAREGNN* [12] is a camouflage-resistant GNN that enhances the aggregation process with the designed modules against camouflages. *PC-GNN* [13] is a GNN-based imbalanced learning method to solve the class imbalance problem in graph-based fraud detection via resampling. *BWGNN* [10] is a beta wavelet GNN, which can better capture anomaly information on a graph.

3) *Experiment Setup*: We use 1000 diffusion steps for the DDPM-based generators, considering both efficiency and effectiveness. For each heterophilic node, we empirically select 70% of its neighbors as the source nodes and translate their embeddings into anomalous embeddings. We set the length L of the sequence to the average of one-hop neighbors. We train the detection models for 100 epochs by Adam optimizer with a learning rate of 0.01, and save the model with the best macro-F1 on validation data for testing. We use 1% labeled data (both for normal and anomalous nodes) as the training set, the remaining data are split as validation and test sets by a ratio of 1:2. We use the following three metrics to present a comprehensive evaluation: *Macro-F1* is the unweighted mean of the F1-score of two classes. *AUC-ROC* is the area under the ROC curve, referring to the probability that a randomly chosen anomaly receives a higher score than a randomly chosen normal object. *AUC-PR* is the area under the curve of precision against recall at different thresholds. The models are implemented based on PyTorch and DGL. We conduct all the experiments on Ubuntu 20.04 with an Intel Core i9-12900K CPU, an NVIDIA GeForce RTX 3090 GPU, and 64 GB memory.

B. Anomaly Detection Performance

We report the anomaly detection performance of CAGAD and the baselines in Table III, and we have the following

TABLE III
PERFORMANCE COMPARISON BETWEEN CAGAD AND BASELINES

Method	PubMed			T-Finance			Amazon			YelpChi		
	F1	AUC-ROC	AUC-PR	F1	AUC-ROC	AUC-PR	F1	AUC-ROC	AUC-PR	F1	AUC-ROC	AUC-PR
GAT	55.15	53.24	46.24	53.15	52.04	43.10	60.84	73.45	68.97	50.27	50.95	24.06
GIN	59.25	69.48	60.35	58.25	68.86	57.03	68.69	78.83	70.22	57.57	64.73	30.57
GraphSAGE	59.63	66.35	57.63	59.03	66.35	54.95	70.78	75.37	70.10	58.41	67.58	31.92
GWNN	71.64	87.68	76.16	70.64	86.68	71.79	87.01	85.37	79.40	59.10	67.16	31.72
LA-GNN	73.15	89.05	77.35	72.15	88.05	72.92	65.15	84.15	78.26	57.25	68.05	34.57
GCA	74.54	90.28	78.89	73.54	88.28	73.11	66.54	86.62	80.56	59.34	68.28	34.71
GraphConsis	73.39	90.56	78.66	71.73	90.28	74.75	68.59	74.11	68.92	56.79	66.41	37.46
CAREGNN	75.47	90.83	75.98	73.32	90.50	71.95	68.78	88.69	80.47	62.18	75.07	39.26
PC-GNN	63.06	91.37	79.36	62.06	90.76	75.17	79.86	<u>90.40</u>	<u>84.07</u>	59.82	75.47	40.58
BWGNN	<u>84.49</u>	<u>93.51</u>	<u>81.22</u>	<u>84.89</u>	<u>91.15</u>	<u>75.49</u>	<u>90.92</u>	89.45	83.19	<u>67.02</u>	<u>76.95</u>	<u>43.41</u>
CAGAD	86.61	95.52	84.39	87.65	93.34	76.70	92.30	92.43	85.23	68.44	78.67	44.78

Note: The best results are highlighted in bold and the second-best are underlined.

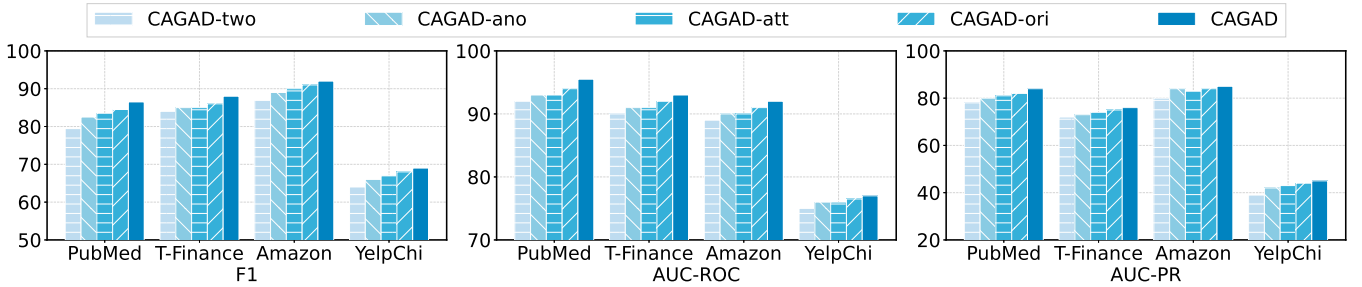


Fig. 5. Ablation study results on four datasets.

observations: 1) For overall detection results, CAGAD yields uniformly better performance than all the baselines across four datasets. In particular, CAGAD obtains on average 2.35%, 2.53%, and 2.79% improvements over the state of the art, BWGNN, in terms of F1, AUC-ROC, and AUC-PR, respectively. 2) The improvements of CAGAD are more significant on T-Finance. We speculate this is because the anomalies in T-Finance are more sparse, and the normal neighbors of anomalies may hinder the vanilla GNN from learning distinguishable representations. Instead, CAGAD can generate augmented counterfactual data to acquire better anomaly representations and further advocate for anomaly detection performance. 3) The two graph data augmentation methods, LA-GNN and GCA, have better performances than the general GNN models (e.g., GraphSAGE and GWNN). These two models are even competitive with the models that are specifically designed for graph anomaly detection (e.g., CAREGNN and PC-GNN). The reason is that only 1% of nodes are adopted as labeled data for model training, and data augmentation can effectively complement the limited training data. This result justifies our motivation that the proposed counterfactual data augmentation can enlarge the training data and be more effective than other augmentation-based or sampling-based GNN methods for anomaly detection.

Not surprisingly, the three anomaly detection baselines, GraphConsis, CAREGNN, and PC-GNN, outperform general GNN models. However, they ignore the problem that the anomalies with numerous benign neighbors might attenuate

their suspiciousness under “vanilla” neighborhood aggregation. BWGNN, which is the best baseline, introduces the graph wavelet theory to remedy this issue, significantly improving the performance compared to others. However, by incorporating counterfactual augmented data, CAGAD can more effectively capture the anomaly information on the graph and outperform BWGNN by a large margin.

C. Ablation Study

Here, we investigate the contributions of the essential components in CAGAD, i.e., the generated anomalous embeddings and the attention mechanism, taking into account different components: 1) *CAGAD-two*, which removes the generated embeddings and the attention mechanism of the model; 2) *CAGAD-ano*, which removes the generated anomalous embeddings produced by G_{ano} during neighborhood aggregation; 3) *CAGAD-att*, which removes the attention mechanism of neighborhood aggregation; and (4) *CAGAD-ori*, which keeps the original embedding from being replaced by the generated one for neighborhood aggregation.

The results are shown in Fig. 5. We summarize our observations as follows.

- 1) When removing the generated embeddings and attention mechanism, *CAGAD-two* becomes a plain GNN and performs the worst, suggesting that without the counterfactual augmented data, traditional neighborhood

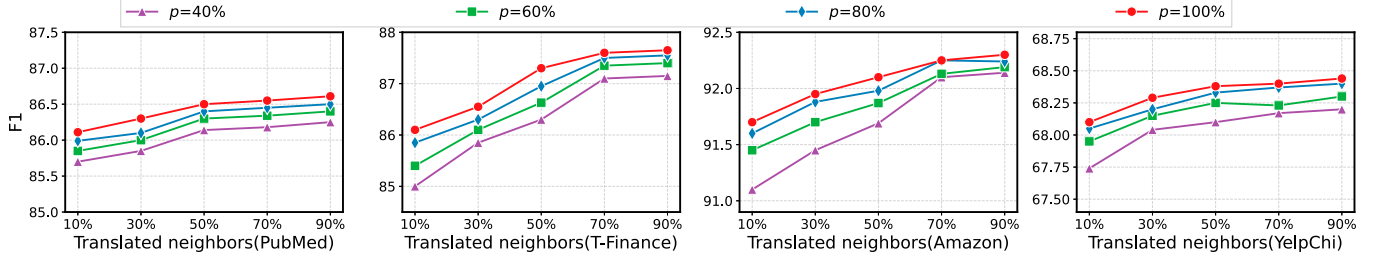


Fig. 6. F1 performance with different numbers of translated neighbors.

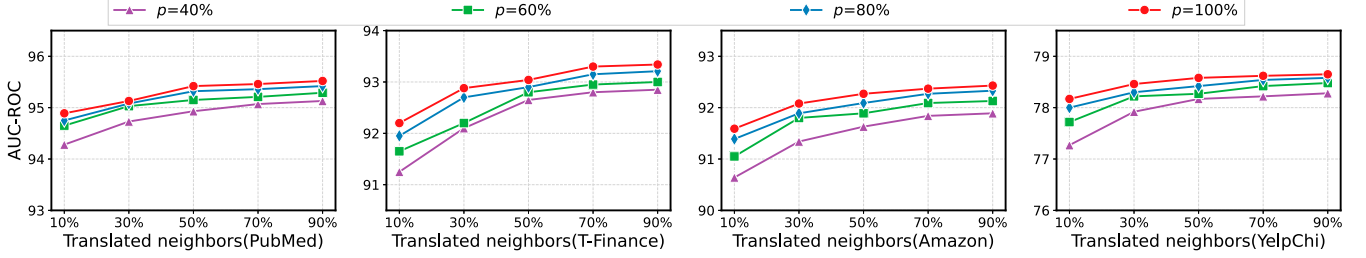


Fig. 7. AUC-ROC performance with different numbers of translated neighbors.

aggregation mechanisms cannot effectively capture anomalous information, resulting in poor performance. Nevertheless, our designed data augmentation method can address this issue and boost detection performance.

- 2) *CAGAD-ano* suffers from significant performance degradation due to the removal of the DDPM-based generator, indicating that involving the generated anomalous embeddings for the neighborhood aggregation yields an obvious performance gain. The generated anomalous embeddings can make the representations of anomalies more distinguishable from normal nodes, and further improve detection performance.
- 3) After eliminating the attention mechanism, the performance of *CAGAD-att* drops significantly, which shows that the attention mechanism can enhance the detection performance. This is particularly true for our case as the counterfactual GNN requires adaptively adjusting the weights of the generated embeddings to obtain better representations. Hence, it is also essential to include the attention mechanism in CAGAD.
- 4) Involving the source embeddings for neighborhood aggregation lowers the performance slightly, which indicates that the designed replacement strategy is effective for anomaly detection. By combining all the components, CAGAD achieves the best performance.

D. Influence of Translated Neighbors

The number of translated neighbors for heterophilic nodes is a critical hyperparameter that affects the distinguishability of the learned representations and, subsequently, influences the anomaly detection performance. We investigate the performance change by altering the number of translated neighbors. In these experiments, for each obtained heterophilic

node, we translate a different proportion of its neighbors into anomalous ones and observe the results when selecting $p = [40, 60, 80, 100]\%$ of all the obtained heterophilic nodes for neighbor manipulation.

The F1 and AUC-ROC of the four datasets are presented in Figs. 6 and 7, respectively. For all the p values, the performance of CAGAD consistently improves with the increase of translated neighbors on all datasets in terms of F1 and AUC-ROC. The performance improvement is more significant for T-Finance and Amazon datasets when we increase the ratio of translated neighbors, as the ratios of anomalies to all neighbors in T-Finance and Amazon datasets are lower. For example, the ratio on Amazon is only 6.8%, while the ratio of anomalies on PubMed is around 20.8%. When there are only a few anomalies, for an anomalous node, the proportion between normal and anomalous neighbors is relatively high. Hence, we need more anomalous neighboring nodes to balance the neighborhoods. This observation demonstrates that our proposed counterfactual data augmentation is more effective on graphs with scarce anomalies. Besides, when the selected neighbor ratio exceeds 70% the performance keeps stable, which indicates that just a given number of anomalous neighboring nodes are required to obtain expected performance. Also, selecting more heterophilic nodes for neighbor manipulation (p) improves anomaly detection performance, which indicates that the heterophilic node detector can accurately identify nodes with heterophily dominant neighbors, and manipulating them can benefit anomaly detection.

E. The Effect of Detection Models

We further investigate the effectiveness of the proposed counterfactual augmentation when applying it to other GNN-based detection methods: GCN and GraphSAGE. We evaluate the

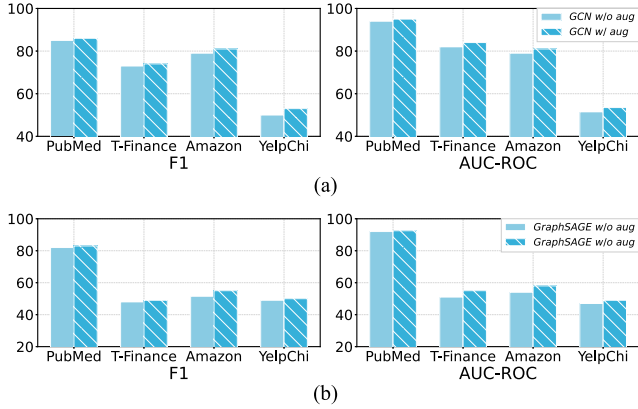


Fig. 8. Effect of the augmented data. (a) Detection performance using GCN model. (b) Detection performance using GraphSAGE model.

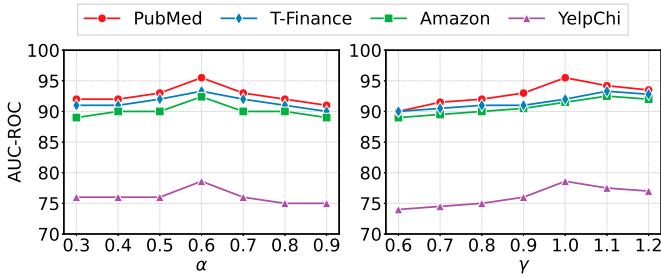


Fig. 9. Effect of the hyperparameters.

model performance with and without the augmented data for graph anomaly detection and denote the two schemes as *ModelName w/ aug* and *ModelName w/o aug*. Fig. 8 shows the F1 and AUC-ROC for the GCN and GraphSAGE models on four datasets. The augmented data are generated by our designed DDPM-based generator. We observe that for both the GCN and GraphSAGE models, the one with the augmented data performs better. For example, the AUC of *GCN w/ aug* is around 2.6% higher than that of *GCN w/o aug* on average. The results suggest that the augmented data generated by our method can improve the detection performance, and the proposed counterfactual data augmentation strategy can be applied to other graph anomaly detection frameworks.

F. Hyperparameter Sensitivity

We now evaluate the impact of two important hyperparameters α and γ . In the heterophilic node detector, a heterophilic node is identified by comparing its heterophily degree h_i^d with α , which directly influences the results of the identification. As shown on the left of Fig. 9, the detection performance improves with the increase of α and then starts to decrease when $\alpha > 0.6$. When α is too small, many nodes might be incorrectly identified as heterophilic. In contrast, larger α prevents our designed heterophilic node detector from taking effect since too few nodes are identified. This indicates that a proper α can obtain expected identification results. The reason may be attributed to the fact

that for anomaly detection tasks, there are only two categories, making the identification of heterophilic nodes feasible. The weight parameter γ in (15) is used to scale the different anomaly features for embedding generation. As shown in the right of Fig. 9, a higher value of γ around 1.1 is preferable to obtain a good performance since higher γ enables the generator to excessively amplify the anomalous characteristics in the generated embeddings.

V. RELATED WORK

This work is mainly related to three research areas: graph anomaly detection, graph data augmentation, and diffusion probabilistic models. Here, we will present an overview of the most closely related works in each area and highlight the major differences between our study and these works.

A. Anomaly Detection on Graphs

Since graph-structured data are ubiquitous and have the capacity to model a wide range of real-world complex systems, identifying anomalies in graphs has drawn increased research interest [38], [39], [40]. Due to their demonstrated superior modeling power for graphs, various GNN-based methods have been proposed to detect anomalies on graphs. The pioneer used GNNs to build an autoencoder to reconstruct the attribute and structure information simultaneously, and the abnormality is evaluated by reconstruction errors [41]. Based on this framework, a tailored deep GCNN is designed to detect local, global, and structural anomalies by capturing community structure in the graph [42]. Contrastive learning and self-supervised learning with GNNs are also introduced to identify the anomalies in attributed networks [23], [43]. Metalearning and hypersphere learning are incorporated into GNNs to leverage the labeled samples for anomaly detection [44], [45], [46], [47].

To remedy the problem that numerous neighbors with normal labels might make the anomaly representations learned by GNNs less distinguishable, multiple resampling (e.g., over-sampling and under-sampling) strategies are designed in [12], [13], and [14]. Researchers also utilized reweighting methods to assign different weights to different samples [15], [16], [17]. More recently, spectral filters are explored to enhance the expressive power of GNNs for learning better anomaly representations [10], [11].

The resampling and reweighting strategies generally require a number of labeled samples and can only manipulate the training data for building a robust model and ignore the manipulation of test data where there still exists the phenomenon of imbalanced neighbors. We also aim to change the neighbors of anomalies to enhance node representation but with a more proper neighborhood generation process for learning anomaly representations instead of sampling existing ones.

B. Graph Data Augmentation

These techniques aim to generate extra data by applying label-preserving transformations on inputs to improve the

model's generalization capability. A widely used scheme is related to edge operations, such as edge dropping and subgraph sampling [48], [49]. One of the early works [50] randomly drops a fixed fraction of edges to generate new graph views for node classification. Following this, task-irrelevant edges are identified and removed by the MLP-based graph sparsification model [51] and the nuclear norm regularization loss [52] to improve the generalization performance. These augmentation methods are also applied to contrastive learning and self-supervised learning for node classification and anomaly detection [22], [53]. Recently, feature augmentation has been adopted for graph data augmentation, which aims to improve the node feature quality by learning additional task-relevant features. Feature augmentation is generally utilized to initiate node features on plain graphs to smoothly incorporate into GNN models and supplement additional node features that are hard to capture by downstream models [37], [54]. Besides, counterfactual data augmentation is exploited to improve the performance of node classification and link prediction [55], [56], [57]. The researchers either generate counterfactual data by injecting interventions on the sensitive attribute of nodes [55], [57] or find out counterfactual links which are the most similar node pairs with different treatments (neighborhood relations) [56] to boost the performance.

The above-mentioned graph data augmentation methods focus on increasing training set by creating label-preserving data or seeking existing counterfactual samples. By contrast, we aim to alleviate the problem that GNNs produce indistinguishable representations for extremely imbalanced node distribution in anomaly detection. Correspondingly, our model translates normal nodes into the ones with opposite labels and steers GNN neighborhood aggregation to manufacture effective counterfactual data. Also, our method can produce counterfactual data in an unsupervised way, which can be applied to test data for enhancing node representations.

C. Diffusion Probabilistic Models

Diffusion probabilistic models aim to generate high-quality data samples by reversing the diffusion process via a Markov chain with discrete timesteps [21] and have acquired state-of-the-art generation instances for many real-world applications. For example, image generation is one of the major applications where diffusion models are exploited to produce high-quality images [28], [58], speed up the sampling process [25], [59], conduct image super-resolution [60], [61], and image-to-image translation [29], [62]. Waveform synthesis is another main application where the diffusion models are utilized to manufacture time-domain speech audio from the prior noise [63], [64]. Diffusion models have also been applied to voice conversion [65], shape generation [66], and time series forecasting [67]. Different from these methods that use diffusion models for regular data types (e.g., image and waveforms), we present a graph-specific diffusion model, which can iteratively extract distinct features of anomalies and exert them on the generative process to manufacture anomalous nodes.

VI. CONCLUSION

In this article, we proposed a CAGAD that can produce counterfactual data by steering the process of GNN neighborhood aggregation. In this framework, we designed a graph-specific diffusion model, which can translate a node embedding to the one with the anomalous label. This model was incorporated into GNN neighborhood aggregation to build a counterfactual GNN, which can learn distinguishable node representations to boost anomaly detection performance. The experiments on four real-world datasets showed that CAGAD achieved state-of-the-art performance compared to many strong baselines, and the counterfactual graph augmentation strategy can be applied to other anomaly detection frameworks.

There were two limitations to our proposed model, which can become fruitful directions for further investigation. First, our counterfactual data augmentation method was only applicable to binary classification scenarios. Hence, we are interested in extending the model to accommodate situations involving multiclass classification. Second, we only investigated the effectiveness of static graphs. Thus, more evaluation is needed to understand the feasibility in the context of more complex graph data, e.g., heterogeneous graphs, spatial-temporal graphs, and dynamic graphs.

REFERENCES

- [1] X. Ma et al., "A comprehensive survey on graph anomaly detection with deep learning," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12012–12038, Dec. 2023.
- [2] K. Liu et al., "Bond: Benchmarking unsupervised outlier node detection on static attributed graphs," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 35, 2022, pp. 27021–27035.
- [3] X. Shen, W. Lv, J. Qiu, A. Kaur, F. Xiao, and F. Xia, "Trust-aware detection of malicious users in dating social networks," *IEEE Trans. Comput. Social Syst.*, vol. 10, no. 5, pp. 2587–2598, Oct. 2023.
- [4] Y. Yang, Y. Xu, Y. Sun, Y. Dong, F. Wu, and Y. Zhuang, "Mining fraudsters and fraudulent strategies in large-scale mobile social networks," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 1, pp. 169–179, Jan. 2021.
- [5] J. Cui, C. Yan, and C. Wang, "ReMEMBER: Ranking metric embedding-based multicontextual behavior profiling for online banking fraud detection," *IEEE Trans. Comput. Social Syst.*, vol. 8, no. 3, pp. 643–654, Jun. 2021.
- [6] W. Hu, Y. Yang, J. Wang, X. Huang, and Z. Cheng, "Understanding electricity-theft behavior via multi-source data," in *Proc. Web Conf.*, 2020, pp. 2264–2274.
- [7] H. Chen, H. Yin, T. Chen, Q. V. H. Nguyen, W.-C. Peng, and X. Li, "Exploiting centrality information with graph convolutions for network representation learning," in *Proc. IEEE Int. Conf. Data Eng.*, 2019, pp. 590–601.
- [8] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *Proc. Int. Conf. Learn. Representations*, 2019.
- [9] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018.
- [10] J. Tang, J. Li, Z. Gao, and J. Li, "Rethinking graph neural networks for anomaly detection," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2022, pp. 21076–21089.
- [11] Z. Chai et al., "Can abnormality be detected by graph neural networks?" in *Proc. Int. Joint Conf. Artif. Intell.*, 2022, pp. 1945–1951.
- [12] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, "Enhancing graph neural network-based fraud detectors against camouflaged fraudsters," in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2020, pp. 315–324.
- [13] Y. Liu et al., "Pick and choose: A GNN-based imbalanced learning approach for fraud detection," in *Proc. Web Conf.*, 2021, pp. 3168–3177.

- [14] Z. Liu, Y. Dou, P. S. Yu, Y. Deng, and H. Peng, "Alleviating the inconsistency problem of applying graph neural network to fraud detection," in *Proc. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2020, pp. 1569–1572.
- [15] D. Wang et al., "A semi-supervised graph attentive network for financial fraud detection," in *Proc. SIAM Int. Conf. Data Mining*, 2019, pp. 598–607.
- [16] L. Cui, H. Seo, M. Tabar, F. Ma, S. Wang, and D. Lee, "DETERRENT: Knowledge guided graph attention network for detecting healthcare misinformation," in *Proc. ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2020, pp. 492–502.
- [17] C. Liu, L. Sun, X. Ao, J. Feng, Q. He, and H. Yang, "Intention-aware heterogeneous graph attention networks for fraud transactions detection," in *Proc. ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2021, pp. 3280–3288.
- [18] G. Pang, C. Shen, L. Cao, and A. V. D. Hengel, "Deep learning for anomaly detection: A review," *ACM Comput. Surveys*, vol. 54, no. 2, pp. 1–38, 2021.
- [19] D. He, C. Liang, H. Liu, M. Wen, P. Jiao, and Z. Feng, "Block modeling-guided graph convolutional neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 4022–4029.
- [20] T. Yang, Y. Wang, Z. Yue, Y. Yang, Y. Tong, and J. Bai, "Graph pointer neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 8832–8839.
- [21] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 6840–6851.
- [22] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, and L. Wang, "Graph contrastive learning with adaptive augmentation," in *Proc. Web Conf.*, 2021, pp. 2069–2080.
- [23] Y. Zheng, M. Jin, Y. Liu, L. Chi, K. T. Phan, and Y.-P. P. Chen, "Generative and contrastive self-supervised learning for graph anomaly detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12220–12233, Dec. 2023.
- [24] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli, "Deep unsupervised learning using nonequilibrium thermodynamics," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 2256–2265.
- [25] Z. Xiao, K. Kreis, and A. Vahdat, "Tackling the generative learning trilemma with denoising diffusion GANs," in *Proc. Int. Conf. Learn. Representations*, 2022.
- [26] Q. Gao, F. Zhou, K. Zhang, F. Zhang, and G. Trajcevski, "Adversarial human trajectory learning for trip recommendation," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 4, pp. 1764–1776, Apr. 2023.
- [27] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.
- [28] C. Meng, Y. Song, J. Song, J. Wu, J.-Y. Zhu, and S. Ermon, "SDEdit: Image synthesis and editing with stochastic differential equations," in *Proc. Int. Conf. Learn. Representations*, 2022.
- [29] A. Sinha, J. Song, C. Meng, and S. Ermon, "D2c: Diffusion-decoding models for few-shot conditional generation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 12533–12548.
- [30] C. Xiao, Z. Gou, W. Tai, K. Zhang, and F. Zhou, "Imputation-based time-series anomaly detection with conditional weight-incremental diffusion models," in *Proc. ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2023, pp. 2742–2751.
- [31] X. Xu, F. Zhou, K. Zhang, and S. Liu, "CCGL: Contrastive cascade graph learning," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 5, pp. 4539–4554, May 2022.
- [32] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher, and T. Eliassi-Rad, "Collective classification in network data," *AI Mag.*, vol. 29, no. 3, pp. 93–93, 2008.
- [33] J. J. McAuley and J. Leskovec, "From amateurs to connoisseurs: Modeling the evolution of user expertise through online reviews," in *Proc. Web Conf.*, 2013, pp. 897–908.
- [34] S. Rayana and L. Akoglu, "Collective opinion spam detection: Bridging review networks and metadata," in *Proc. ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2015, pp. 985–994.
- [35] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1025–1035.
- [36] B. Xu, H. Shen, Q. Cao, Y. Qiu, and X. Cheng, "Graph wavelet neural network," in *Proc. Int. Conf. Learn. Representations*, 2019.
- [37] S. Liu et al., "Local augmentation for graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 14054–14072.
- [38] Y. Gao, X. Wang, X. He, Z. Liu, H. Feng, and Y. Zhang, "Addressing heterophily in graph anomaly detection: A perspective of graph spectrum," in *Proc. ACM Web Conf.*, 2023, pp. 1528–1538.
- [39] L. He, G. Xu, S. Jameel, X. Wang, and H. Chen, "Graph-aware deep fusion networks for online spam review detection," *IEEE Trans. Comput. Social Syst.*, vol. 10, no. 5, pp. 2557–2565, Oct. 2023.
- [40] X. Li, C. Xiao, Z. Feng, S. Pang, W. Tai, and F. Zhou, "Controlled graph neural networks with denoising diffusion for anomaly detection," *Expert Syst. Appl.*, vol. 237, 2024, Art. no. 121533.
- [41] K. Ding, J. Li, R. Bhanushali, and H. Liu, "Deep anomaly detection on attributed networks," in *Proc. SIAM Int. Conf. Data Mining*, 2019, pp. 594–602.
- [42] X. Luo et al., "ComGA: Community-aware attributed graph anomaly detection," in *Proc. ACM Int. Conf. Web Search Data Mining*, 2022, pp. 657–665.
- [43] J. Zhang, S. Wang, and S. Chen, "Reconstruction enhanced multi-view contrastive learning for anomaly detection on attributed networks," in *Proc. Int. Joint Conf. Artif. Intell.*, 2022, pp. 2376–2382.
- [44] K. Ding, Q. Zhou, H. Tong, and H. Liu, "Few-shot network anomaly detection via cross-network meta-learning," in *Proc. Web Conf.*, 2021, pp. 2448–2456.
- [45] L. Ruff et al., "Deep semi-supervised anomaly detection," in *Proc. Int. Conf. Learn. Representations*, 2020.
- [46] A. Kumagai, T. Iwata, and Y. Fujiwara, "Semi-supervised anomaly detection on attributed graphs," in *Proc. Int. Joint Conf. Neural Netw.*, 2021, pp. 1–8.
- [47] S. Zhou, X. Huang, N. Liu, Q. Tan, and F.-L. Chung, "Unseen anomaly detection on networks via multi-hypersphere learning," in *Proc. SIAM Int. Conf. Data Mining*, 2022, pp. 262–270.
- [48] K. Ding, Z. Xu, H. Tong, and H. Liu, "Data augmentation for deep graph learning: A survey," *ACM SIGKDD Explorations Newslett.*, vol. 24, no. 2, pp. 61–77, 2022.
- [49] T. Zhao, Y. Liu, L. Neves, O. Woodford, M. Jiang, and N. Shah, "Data augmentation for graph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 11015–11023.
- [50] Y. Rong, W. Huang, T. Xu, and J. Huang, "DropEdge: Towards deep graph convolutional networks on node classification," in *Proc. Int. Conf. Learn. Representations*, 2020.
- [51] C. Zheng et al., "Robust graph representation learning via neural sparsification," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 11458–11468.
- [52] D. Luo et al., "Learning to drop: Robust graph neural network via topological denoising," in *Proc. Int. Conf. Web Search Data Mining*, 2021, pp. 779–787.
- [53] Y. Liu, Z. Li, S. Pan, C. Gong, C. Zhou, and G. Karypis, "Anomaly detection on attributed networks via contrastive self-supervised learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 6, pp. 2378–2392, Jun. 2022.
- [54] K. Kong et al., "Robust optimization as data augmentation for large-scale graphs," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 60–69.
- [55] J. Ma, R. Guo, M. Wan, L. Yang, A. Zhang, and J. Li, "Learning fair node representations with graph counterfactual fairness," in *Proc. Int. Conf. Web Search Data Mining*, 2022, pp. 695–703.
- [56] T. Zhao, G. Liu, D. Wang, W. Yu, and M. Jiang, "Learning from counterfactual links for link prediction," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 26911–26926.
- [57] C. Xiao, X. Xu, Y. Lei, K. Zhang, S. Liu, and F. Zhou, "Counterfactual graph learning for anomaly detection on attributed networks," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 10, pp. 10540–10553, Oct. 2023.
- [58] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," in *Proc. Int. Conf. Learn. Representations*, 2021.
- [59] T. Dockhorn, A. Vahdat, and K. Kreis, "Score-based generative modeling with critically-damped Langevin diffusion," in *Proc. Int. Conf. Learn. Representations*, 2022.
- [60] C. Saharia, J. Ho, W. Chan, T. Salimans, D. J. Fleet, and M. Norouzi, "Image super-resolution via iterative refinement," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 4, pp. 4713–4726, Apr. 2023.
- [61] H. Li et al., "SRDiff: Single image super-resolution with diffusion probabilistic models," *Neurocomputing*, vol. 479, pp. 47–59, 2022.
- [62] H. Sasaki, C. G. Willcocks, and T. P. Breckon, "UNIT-DDPM: Unpaired image translation with denoising diffusion probabilistic models," 2021, *arXiv:2104.05358*.
- [63] N. Chen, Y. Zhang, H. Zen, R. J. Weiss, M. Norouzi, and W. Chan, "WaveGrad: Estimating gradients for waveform generation," in *Proc. Int. Conf. Learn. Representations*, 2021.
- [64] M. W. Lam, J. Wang, D. Su, and D. Yu, "BDDM: Bilateral denoising diffusion models for fast and high-quality speech synthesis," in *Proc. Int. Conf. Learn. Representations*, 2022.

- [65] V. Popov, I. Vovk, V. Gogoryan, T. Sadekova, M. Kudinov, and J. Wei, "Diffusion-based voice conversion with fast maximum likelihood sampling scheme," in *Proc. Int. Conf. Learn. Representations*, 2022.
- [66] L. Zhou, Y. Du, and J. Wu, "3D shape generation and completion through point-voxel diffusion," in *Proc. Int. Conf. Comput. Vis.*, 2021, pp. 5826–5835.
- [67] K. Rasul, C. Seward, I. Schuster, and R. Vollgraf, "Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 8857–8868.



Chunjing Xiao received the Ph.D. degree in computer software and theory from the University of Electronic Science and Technology of China, Chengdu, Sichuan, China, in 2013.

He is currently an Associate Professor with the School of Computer and Information Engineering, Henan University, Kaifeng, China. He was a Visiting Scholar with the Department of Electrical Engineering and Computer Science, Northwestern University, Evanston, IL, USA. His research interests include anomaly detection, recommender systems, and Internet of Things.



Shikang Pang received the B.E. degree in software engineering from the School of Computer and Information Engineering, Huanghuai University, Zhumadian, China, in 2019. He is currently working toward the M.S. degree in computer science with the School of Computer and Information Engineering, Henan University, Kaifeng, China.

His research interests include anomaly detection, data analytics, and social network data mining.



Xovee Xu (Graduate Student Member, IEEE) received the B.S. and M.S. degrees in software engineering from the University of Electronic Science and Technology of China (UESTC), Chengdu, Sichuan, China, in 2018 and 2021, respectively, where he is currently working toward the Ph.D. degree in computer science.

His research interests include social network data mining and knowledge discovery, primarily focuses on information diffusion in full-scale graphs, human-centered data mining, representation learning, and their novel applications in various social and scientific scenarios.



Xuan Li received the B.S. and M.S. degrees in software engineering from Sichuan University, Chengdu, Sichuan, China, in 2009 and 2012, respectively, where he is currently working toward the Ph.D. degree in electronic information.

He is currently a Teacher with Sichuan Post and Telecommunication College, Chengdu, Sichuan, China. His research interests include anomaly detection, data analytics, social network data mining, and knowledge discovery.



Goce Trajceviski (Member, IEEE) received the B.Sc. degree in informatics and automation from the University of Sts. Kiril i Metodij, Skopje, North Macedonia, in 1989, and the M.S. and Ph.D. degrees in computer science from the University of Illinois at Chicago, Chicago, IL, USA, in 1995 and 2002, respectively.

He is currently an Associate Professor with the Department of Electrical and Computer Engineering, Iowa State University, Ames, IA, USA. His research has been funded by the NSF, ONR, BEA, and Northrop Grumman Corporation. In addition to a book chapter and three encyclopedia chapters, he has coauthored over 140 publications in refereed conferences and journals. His research interests include the areas of spatiotemporal data management, uncertainty and reactive behavior management in different application settings, and incorporating multiple contexts.

Dr. Trajceviski was the General Co-Chair of the IEEE International Conference on Data Engineering 2014 and ACM SIGSPATIAL 2019, the PC Co-Chair of the ADBIS 2018 and ACM SIGSPATIAL 2016 and 2017, and has served in various roles in organizing committees in numerous conferences and workshops. He is an Associate Editor of *ACM Transactions on Spatial Algorithms and Systems* and *Geoinformatica* journals.



Fan Zhou (Member, IEEE) received the B.S. degree in computer science from Sichuan University, Chengdu, China, in 2003, and the M.S. and Ph.D. degrees in computer science from the University of Electronic Science and Technology of China, Chengdu, Sichuan, China, in 2006 and 2012, respectively, where he is currently a Professor with the School of Information and Software Engineering.

His research interests include machine learning, neural networks, spatiotemporal data management, graph learning, recommender systems, and social

network data mining.